Some More Flutter Concepts

Flutter

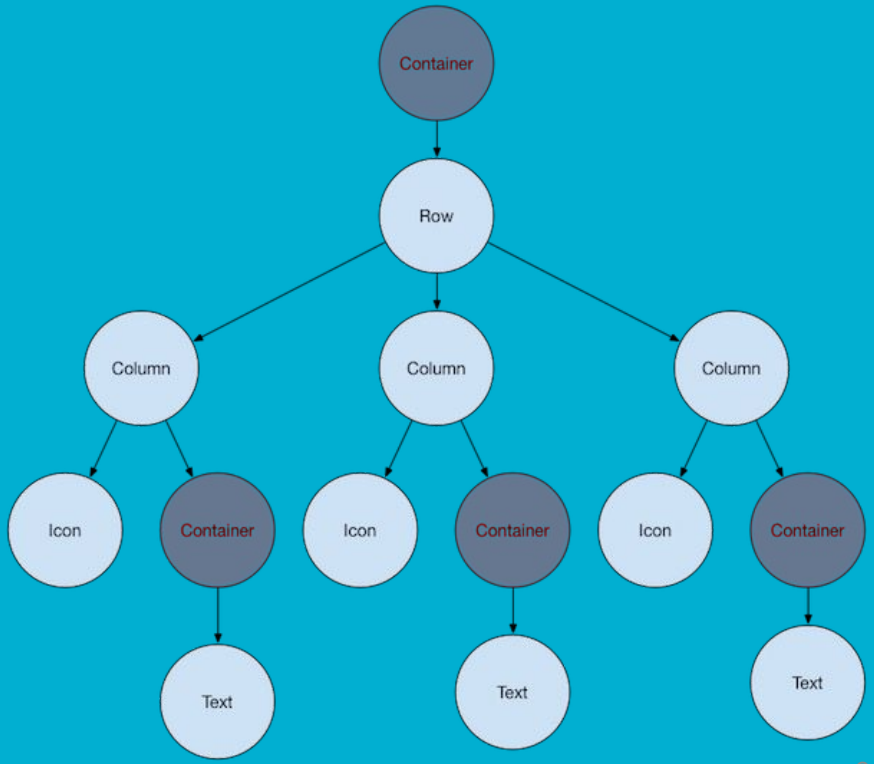
IS2205 - Mobile Application Design & Development

Overview

- Widgets
- Hierarchy
- Types of Widgets
- Stateful Widgets and Stateless Widgets
- Some layout widgets
- Activity

Widgets : A Tree

- Flutter UI components are called **Widgets.**
- Widgets are built by composing other widgets, progressively from more basic widgets.
- i.e.: **Padding** is a widget rather than a **property** of other widgets.
- Therefore UIs consist of many, many widgets.



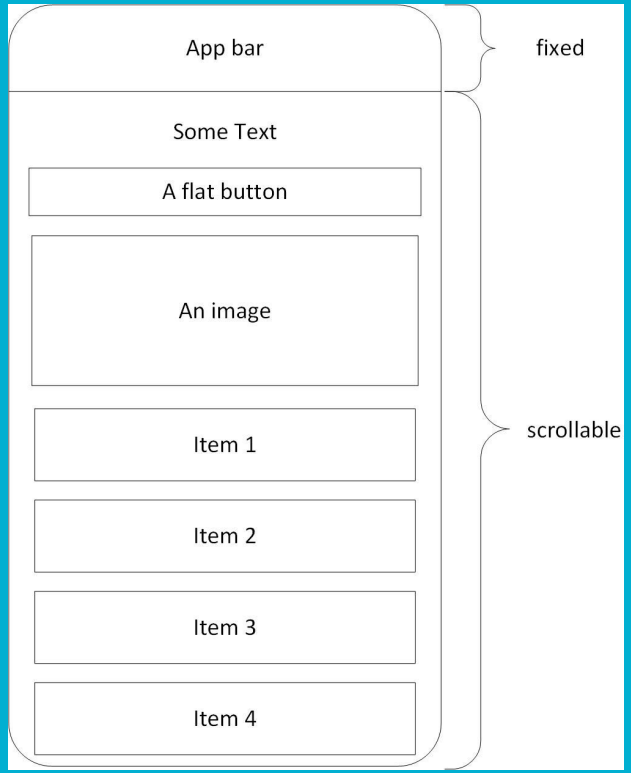
Widgets Composition Flutter vs. Android:

Comparison

- **Flutter Widgets:** Flutter widgets are composable and can be nested within each other to create complex UI layouts. The entire UI is built using a widget tree where each widget represents a part of the UI.
- **Android Views:** Android Views are two fold:

  Composables : A flutter like approach

  XML layouts : Using **ViewGroup** containers.



Declarative UI & State Management

Imperative(XML):textView = findViewById(R.id.title) textView.text = "Hello"

Declarative (Compose & Flutter): Describe UI for given state. Framework rebuilds.

function(State) => UI

Flutter widgets are built using a modern framework that that is aligned with Jetpack's Composables than Android's older XML definitions.

Widgets describe what their view should look like given their current configuration and state.

Fate of Activities

The entire application runs inside a single native container (FlutterActivity).

Does not create new Activities for different screens.

Widgets in Flutter manage their own routing, state, and rendering within that single native Activity window.

Jetpack Compose:	Activity --> setContent --> Composable Tree
Flutter:	Activity --> runApp() --> Widget Tree

Widget Tree and Element Tree

- The **element tree** is an internal representation **created from** the **widget tree**.
- Elements are associated with widgets but are more optimized for performance.
- Elements have a one-to-one correspondence with widgets.
- Widgets are blueprints or templates for the UI components, whereas elements are instances used for rendering.
- Elements have additional features for managing state and lifecycle compared to widgets.

Widget Tree and Element Tree…

Basic Reconciliation Process:

- When the UI needs to update due to changes in application state, Flutter performs a reconciliation process.
- During reconciliation, the flutter framework compares the previous widget tree with the new one to determine the differences.
- It then updates the element tree accordingly.
- This can minimize the number of changes and redraws.

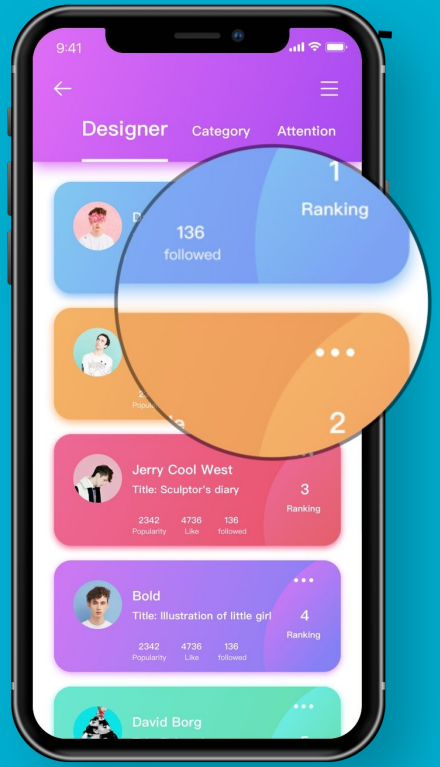
Widgets

There are four main categories of Widgets in Flutter.
Value – Checkbox, FlatButton. Icon, Text, Radio, Image etc. (these Widgets hold a 'Value')

Layout – Align, AppBar, ButtonBar, Center, Column, GridView etc. (these Widgets enables controlling the look of each view).

Navigation – AlertDialog, Drawer, TabBar, Navigator, SimpleDialog etc. (these Widgets are used when the app needs more than one view and a way to navigate users to different views within the app.

Other – other category is for all Widgets that do not fall into the above categories. Such as GestureDetecor, Dismissible, Theme etc.



Stateless and Stateful Widgets

Widgets in Flutter can be categorized into two main types: **stateless** and **stateful**.

Stateless Widgets:

Stateless widgets are immutable : their properties do not change once they are created.

Used for UI components that don't need to change over time, i.e.: static text labels or icons.

Stateful Widgets:

Stateful widgets are mutable : they can maintain and update their internal state.

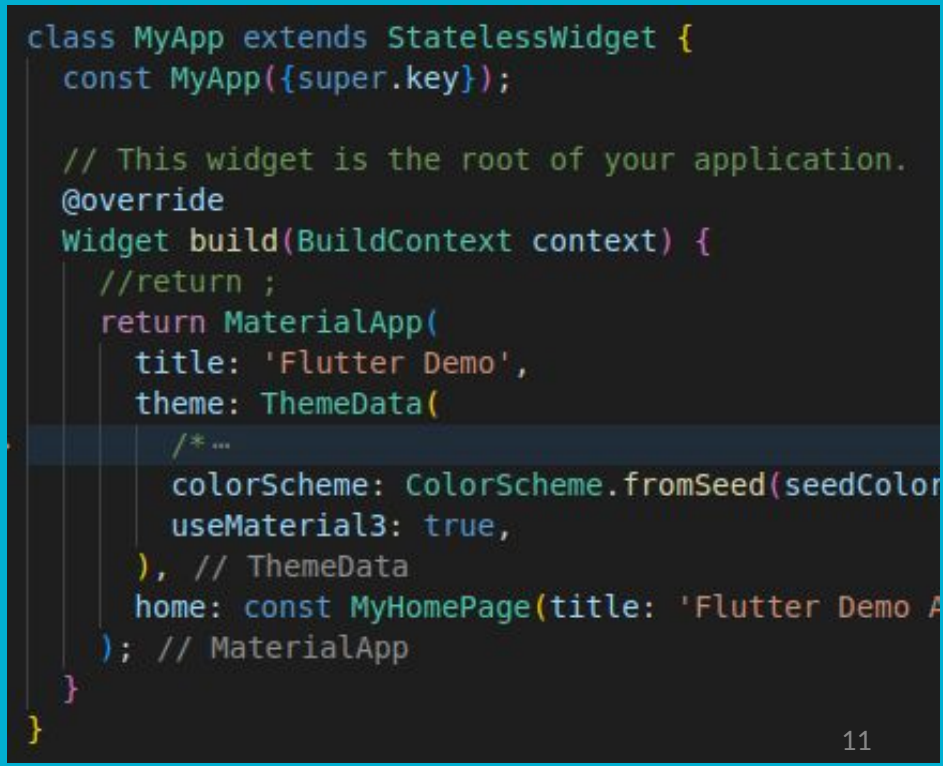
They are used for UI components that can change or respond to user interactions, such as buttons or input fields.

Stateless and Stateful Widgets

Stateless Widget:

In the following the MyApp is a class that extends

StatelessWidget class which returns a MaterialApp widget



Stateless and Stateful Widgets

Stateful Widget:
The State<MyHomePage> class is a generic class that provides the basic functionality for managing the state of a widget.

The _MyHomePageState class extends the State<MyHomePage> class and adds specific functionality for managing the state of the MyHomePage widget.

The _MyHomePageState class also has a **build()** method that builds the user interface for the MyHomePage widget.

```
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});
/*…
  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

  void _incrementCounter() {
    setState(() {
      /*…
      _counter++;
    });
  }
```

Selecting the stateful/stateless

- Prefer stateless widgets over stateful widgets
- Stateless widgets are lighter than the stateful widgets
- If the states need to be maintained code refactoring can be done later

Activity

- List down some performance best practices by reading: https://api.flutter.dev/flutter/widgets/StatefulWidget-class.html

Layout Control

Nesting widgets allows developers to control layout and spacing.

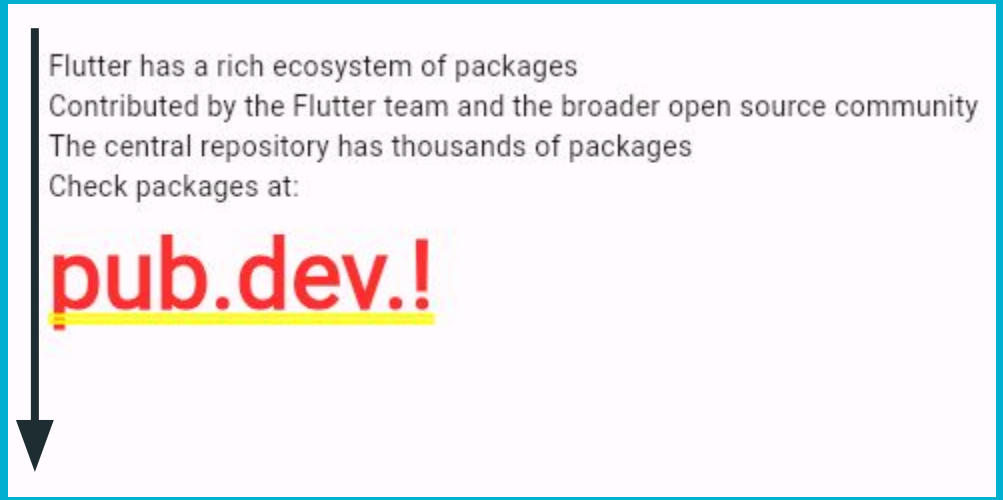
- By nesting widgets like `Row`, `Column` and `Container`you can create UI designs as appropriate.

- Row - A widget that displays its children in a horizontal array.
- Column - A widget that displays its children in a vertical array.
- Container - A widget for laying out other components inside.

Column

```
child: Column(
  mainAxisAlignment: MainAxisAlignment.center, //
  children: [
    //Text('A random \nNew idea:'),
    BigCard(pair: pair),
    SizedBox(
      height: 10,
    ), // SizedBox
    // ↓ Add this.
    Row(
      mainAxisSize: MainAxisSize
          .min, //Minimize the amount of free spa
      children: [
        // ↓ And this.
```

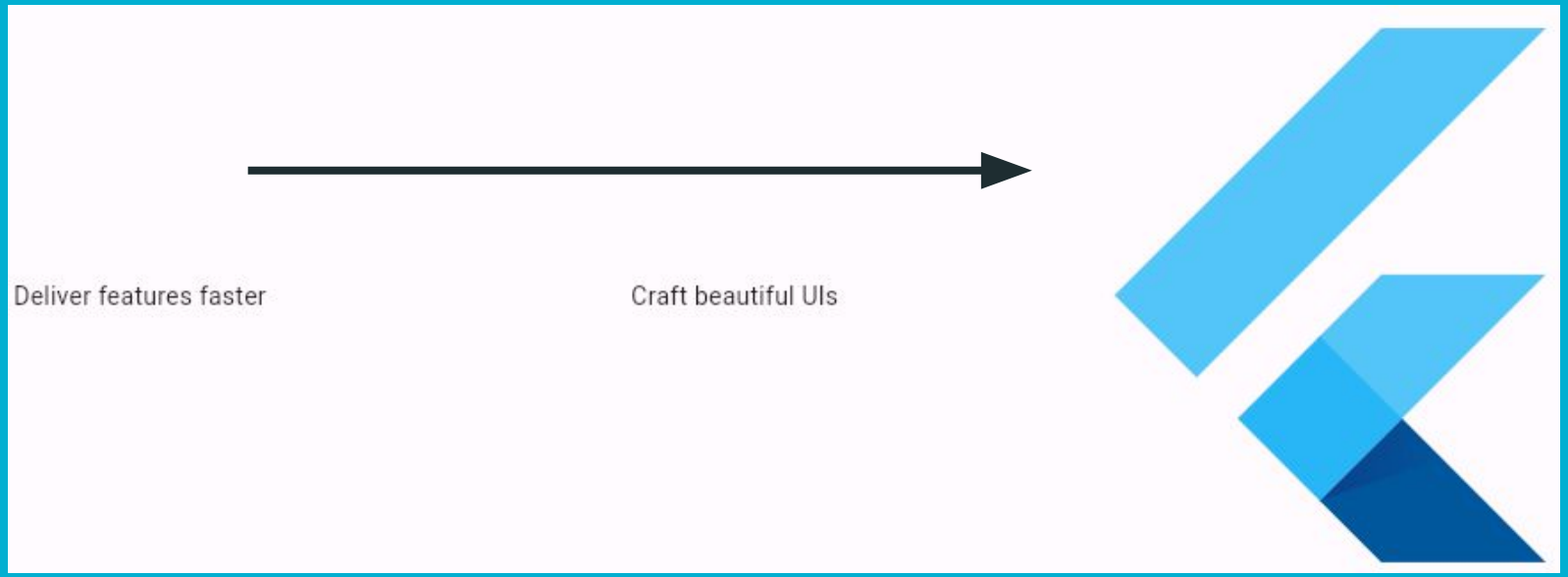
Column


Flutter has a rich ecosystem of packages
Contributed by the Flutter team and the broader open source community
The central repository has thousands of packages
Check packages at:
pub.dev.!

Row

```
Row(
  mainAxisSize: MainAxisSize.min,
  children: [
    // ↓ And this.
    ElevatedButton.icon(
      onPressed: () {
        appState.toggleFavorite();
      },
      icon: Icon(icon),
      label: Text('Like'),
    ), // ElevatedButton.icon
    SizedBox(width: 10),
    ElevatedButton(
      onPressed: () {
```

Row



Container

A **single-child layout widget**.

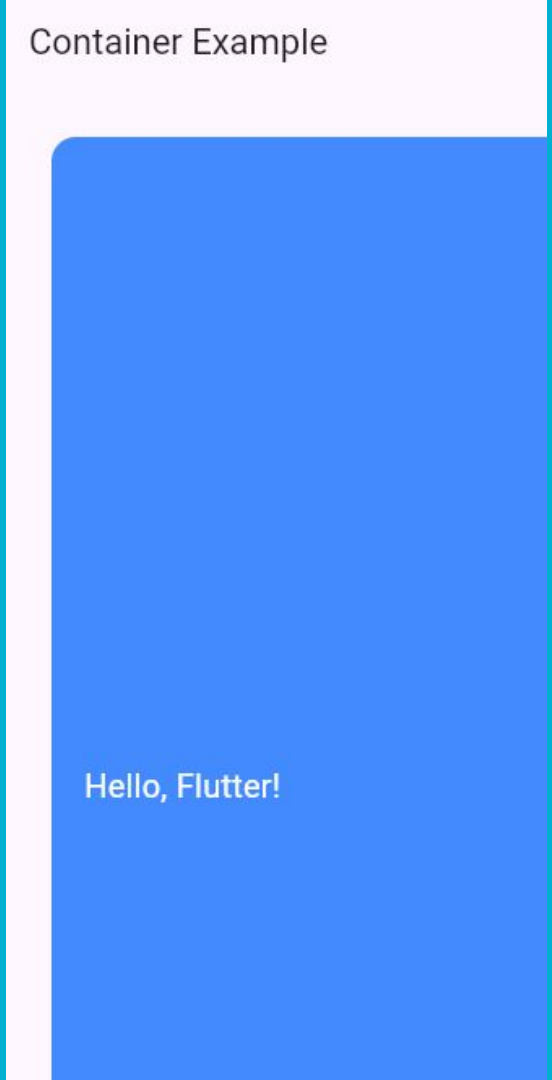
Think of it as a **box** that can style, position, and size **one child widget**.

- Adding **padding** or **margin** around a widget.
- Giving **background color**, **border**, or **rounded corners**.
- Controlling **width** and **height** of a widget.

Container

```
    home: Scaffold(
      appBar: AppBar(title: const Text("Container Example")),
      body: Center(
        child: Container(
          padding: const EdgeInsets.all(20),
          margin: const EdgeInsets.all(30),
          alignment: Alignment.centerLeft, //topLeft/center/centerLeft
          decoration: BoxDecoration(
            color: Colors.blueAccent,
            borderRadius: BorderRadius.circular(15),
          ), // BoxDecoration
          child: const Text(
            "Hello, Flutter!",
            style: TextStyle(fontSize: 20, color: Colors.white),
          ), // Text
        ), // Container
      ), // Center
    ), // Scaffold
  ); // MaterialApp
}
}
```

Text can be positioned inside the container



Layout Activity

Tryout the following tutorial from flutter

https://docs.flutter.dev/ui/layout/tutorial

Layout Activity

Result